Universitatea POLITEHNICA din București
Facultatea de Automatică și Calculatoare

Electronică digitală

Curs – Modulul 11

Circuite secvențiale complexe

dr. ing. Răzvan Tătăroiu
razvan.tataroiu@upb.ro

2022-2023

Sumar

- Circuite logice complexe
  - secvențiale / sincrone – funcționează cu ceas
- Considerații generale:
  - timp / performanță
  - consum energetic
  - suprafață utilizată
- Considerații de implementare:
  - arhitecturi generale
  - tehnologii de realizare

Introducere

Aplicații avute în vedere

- practic orice circuit digital cu rol de calcul, control, monitorizare, prelucrare / procesare
- exemple:
  - microprocesoare / microcontrolere de uz general
    - în sensul proiectării procesorului propriu-zis, la nivel de circuit; nu discutăm *software*-ul ce rulează pe acesta
  - procesoare digitale de semnal
    - audio / video
    - pentru comunicații *wireless*
  - procesoare de uz special
    - criptografie
    - filtrare și rutare de pachete în rețele

Alte exemple

- sisteme de achiziție / control de timp real
  - cerințe de precizie în timp de ordinul **ms**
    - microcontrolere de uz general (sau module ce le integrează) programate corespunzător
      - proiectantul lucrează cu precădere la nivel de ***software embedded***
  - precizie de ordinul **µs**
    - în general **nu** se poate obține exclusiv în *software*
    - microcontroler cu facilități speciale – periferice sincronizate independent de procesor
  - precizie de ordinul **ns**
    - logică dedicată (*custom logic*)
      - proiectantul lucrează cu precădere la nivel de **circuit digital**

Observație

- Ce au în comun:
  - un microprocesor de uz general
  - un procesor specializat
  - o logică dedicată pentru control rapid ?
- Sunt circuite digitale:
  - secvențiale / sincrone (în general)
    - funcționează cu un semnal periodic de ceas
    - produc periodic rezultate noi (ieșiri) pe baza intrărilor precedente și regulilor implementate
  - care trebuie analizate, proiectate, optimizate, documentate și fabricate

Comentarii

- Cursul privește toate aceste aplicații, dar se concentrează pe cazul logicii dedicate
- Exemplele discutate vor fi mult simplificate
  - se poate analiza manual schema la nivel de porți logice a unui procesor modern? În niciun caz – este generată automat pe baza unei descrieri de nivel înalt
  - principiile generale se păstrează

Tehnologii de realizare a circuitelor digitale

Presupunere

- Avem la dispoziție o schemă de nivel jos (care conține doar elemente simple):
  - porți logice
  - bistabile
  - eventual blocuri simple – multiplexoare, decodoare, numărătoare
- versus o schemă de nivel înalt, care conține blocuri complexe, de exemplu unități de procesare – acestea fiind reduse prin procesul de **sinteză** la elemente simple

Variante de implementare

- emulare software
- schemă discretă
- ASIC
- FPGA

Emulare software

- Schema, introdusă într-un format specific, este **interpretată** de un program ce rulează pe un sistem dedicat (de obicei bazat pe microcontroler) dotat cu intrări și ieșiri digitale
  - sau este chiar **tradusă** într-un program
- Performanțe foarte reduse
  - în comparație cu implementările *hardware*
  - timp de răspuns de ordinul 100 ms
- Aplicație tipică:
  - control industrial - *Programmable Logic Controller (PLC)*

Scheme discrete

- Schemă cu porți logice realizate cu relee sau cu tranzistoare discrete
  - se pretează doar pentru scheme foarte simple
  - schemele complexe se implementează în PLC (în loc de relee) sau cu circuite integrate (în loc de tranzistoare discrete)
- Schemă folosind circuite integrate pe scară redusă sau medie *(small-scale integration)*
  - seria 74HC sau serii similare mai noi (74AUP etc.)
  - se pretează pentru scheme relativ simple

Scheme puternic integrate

- la schemele complexe (practic orice implică o procesare semnificativă de date), formate din mii ... milioane de porți logice, este nepractică realizarea discretă
- circuite integrate pe scară foarte largă (*Very Large Scale Integration – VLSI*)
  - implementează întreaga schemă într-un singur circuit integrat (sau un număr redus de cipuri)
  - ex.: un microprocesor

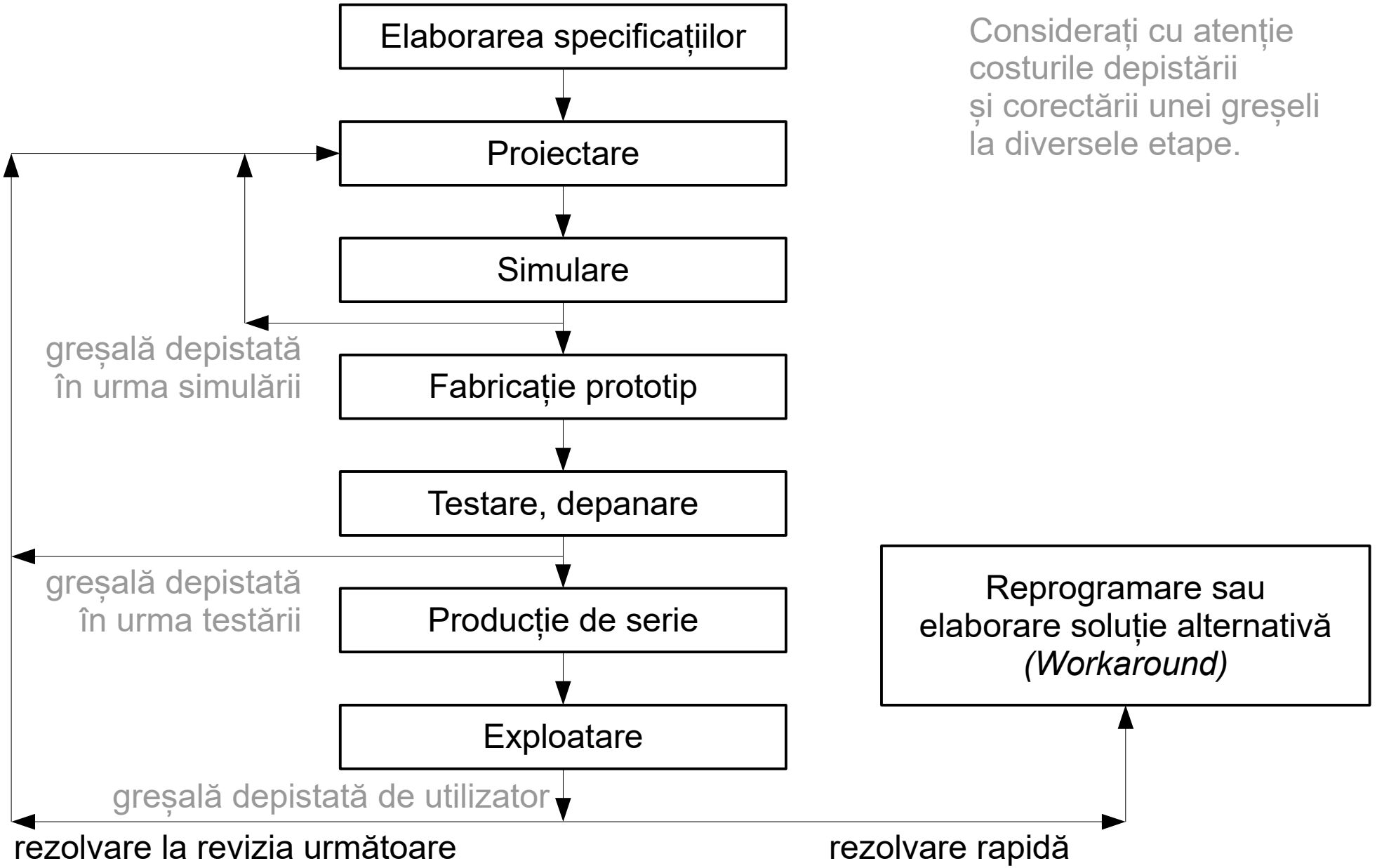
Implementări VLSI

- Circuit integrat fabricat special pentru aplicația avută în vedere, conform schemei date (*Application-Specific Integrated Circuit – ASIC*)
  - cost inițial foarte mare
  - timp de fabricație lung (~ jumătate de an)
- Arie logică programabilă (*Field-Programmable Gate Array – FPGA*)
  - performanță redusă față de ASIC
- Dispozitiv logic programabil (*PLD, CPLD*)
  - similar funcțional cu FPGA dar mult mai simplu

Etape de proiectare *(Workflow)*

Etapă	Rezutat
Elaborarea specificațiilor circuitului (“ce trebuie să facă”)	Document(e) de specificații
Proiectarea circuitului la nivel înalt * descriere structurală * descriere comportamentală	Scheme-bloc de nivel înalt; Cod în limbaj de descriere hardware (Verilog, VHDL etc.)
Sinteza circuitului digital (proces automat, realizat de un sintetizor; analog compilării unui program)	Schemă de nivel jos (*netlist*) conține doar dispozitive elementare: porți, bistabile, eventual blocuri simple
Generarea circuitului digital final (proces de asemenea automat) * conversia schemei (cu porți abstracte) în formă specifică tehnologiei (ex.: porți CMOS / celule logice programabile) * plasarea și interconectarea pieselor (*Place and Route*)	ASIC: fișierele pentru fotolitografie (desenele circuitului integrat) FPGA: fișier de configurare (*bitstream* care se programează în FPGA)

Etape extinse



Considerați cu atenție
costurile depistării
și corectării unei greșeli
la diversele etape.

Considerații de proiectare

Considerații de proiectare

- Legate de performanțele produsului
  - timpul de răspuns / performanța computațională *(latency, throughput)*
  - consumul energetic
  - aria ocupată
- Legate de procesul ingineresc
  - durata și costul procesului de dezvoltare
  - ușurința de adoptare și utilizare (integrare în alte produse)
    - claritatea documentației
    - interfețele *hardware* și *software* oferite

Context

- Considerăm un model simplificat pentru un sistem de calcul, având: o intrare, o ieșire și stare internă (memorie)
  - intrarea și ieșirea reprezintă în general date complexe (grupuri de mai mulți biți)
- Periodic (de obicei sincron cu un semnal global de ceas) sistemul primește noi date de intrare și produce noi date de ieșire

Latență

- *Latency*, ≈ timp de răspuns
  - durata de la primirea unor date noi până la furnizarea rezultatelor corespunzătoare
  - măsurată în unități de timp (μs, ns etc.) sau în număr de perioade ale ceasului (N x T_{CLK})

Throughput

- *Throughput*, ≈ viteză de procesare, performanță computațională
  - viteza cu care sistemul este capabil să furnizeze rezultate noi
  - numărul de rezultate noi furnizate într-un interval specificat de timp

Ilustrare

- Latency vs. Throughput
  - exemplu clasic: comunicația prin rețea cu un server
  - clientul emite o cerere către server
  - serverul răspunde
  - după un anumit timp clientul începe să primească datele – *latency*
  - datele sosesc cu o anumită viteză / frecvență (megabiți / secundă) – *throughput*
  - de obicei *latency* >> 1 / *throughput*
    - atât în rețele cât și în procesoarele moderne; vom explica în cap. urm. de ce apare fenomenul în circuitele digitale

Consumul energetic

- Discutat în cadrul tehnologiei CMOS
- În general există un compromis performanță – consum
- Comporomisul poate fi îmbunătățit prin:
  - optimizări tehnologice – miniaturizare
    - discutat la CMOS
  - optimizări arhitecturale – **paralelizare**
    - discutat în continuare

Aria ocupată de circuit (1)

- Proporțională cu complexitatea circuitului
  - număr de elemente (în principal porți logice)
  - număr de interconexiuni
- Aspect relevant atât pentru
  - ASIC – aria ocupată pe circuitul integrat de tranzistoarele din componența porților logice, respectiv de traseele de legătură
  - FPGA – numărul de blocuri logice configurabile (resurse dinăuntrul FPGA-ului) necesar pentru implementarea schemei – se reduce tot la o arie proporțională cu complexitatea schemei

Aria ocupată de circuit (2)

- Regulă generală: costul crește cu aria
  - ASIC:
    - trebuie fabricat un cip mai mare
    - cipurile se fabrică în paralel pe *wafere* de dimensiune fixă
    - => costul pe cip e aproximativ proporțional cu aria
  - FPGA:
    - în general un FPGA cu mai multe resurse costă mai mult deoarece la rândul său are o arie fizică mai mare
- și consumul static crește în general cu complexitatea circuitului

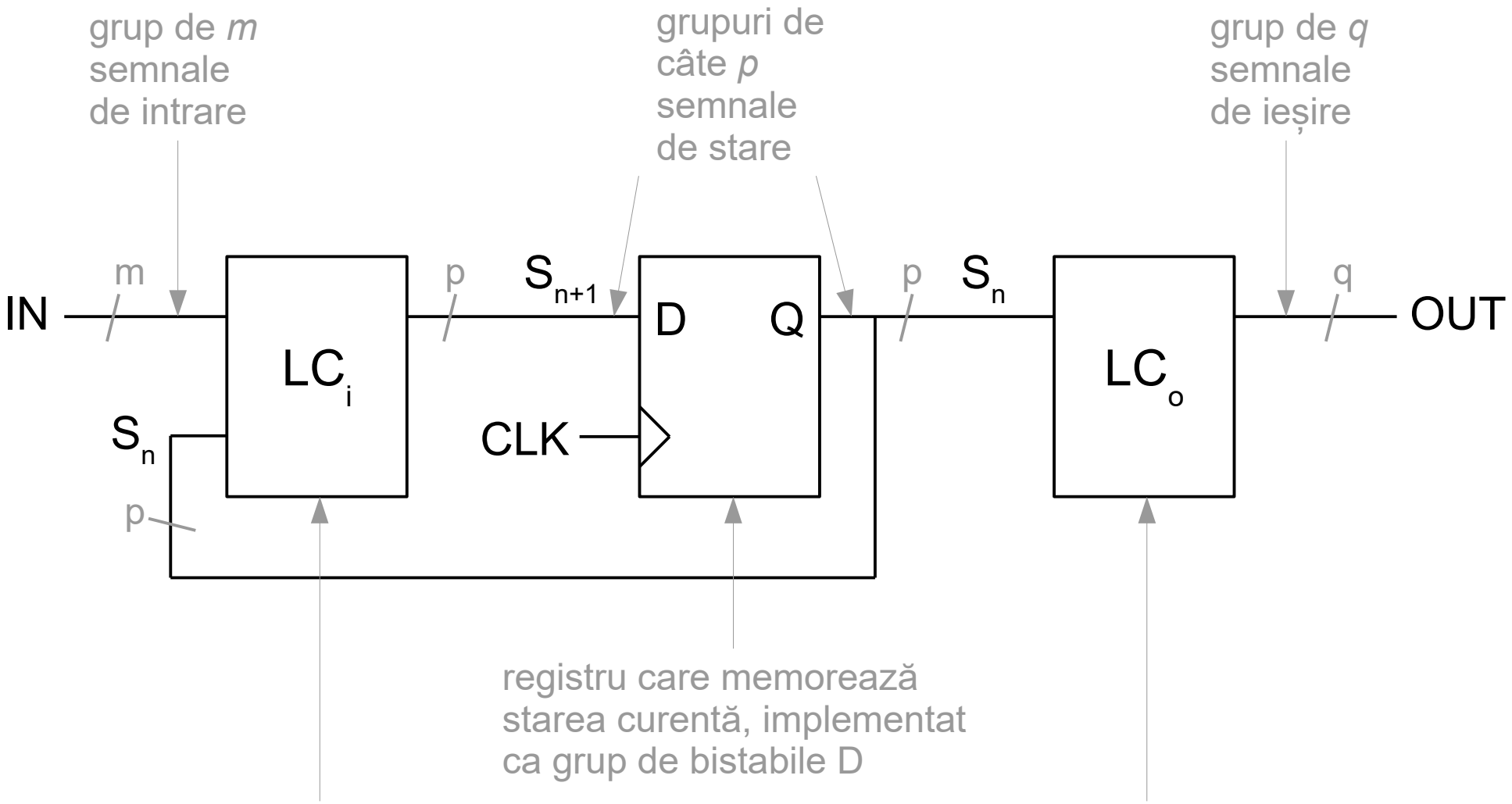
Arhitecturi generale pentru circuitele secvențiale

Arhitecturi

- Automat de stări (*state machine*)
- Bandă de asamblare (*pipeline*)

- Paradigma transferului între registre (*RTL*)

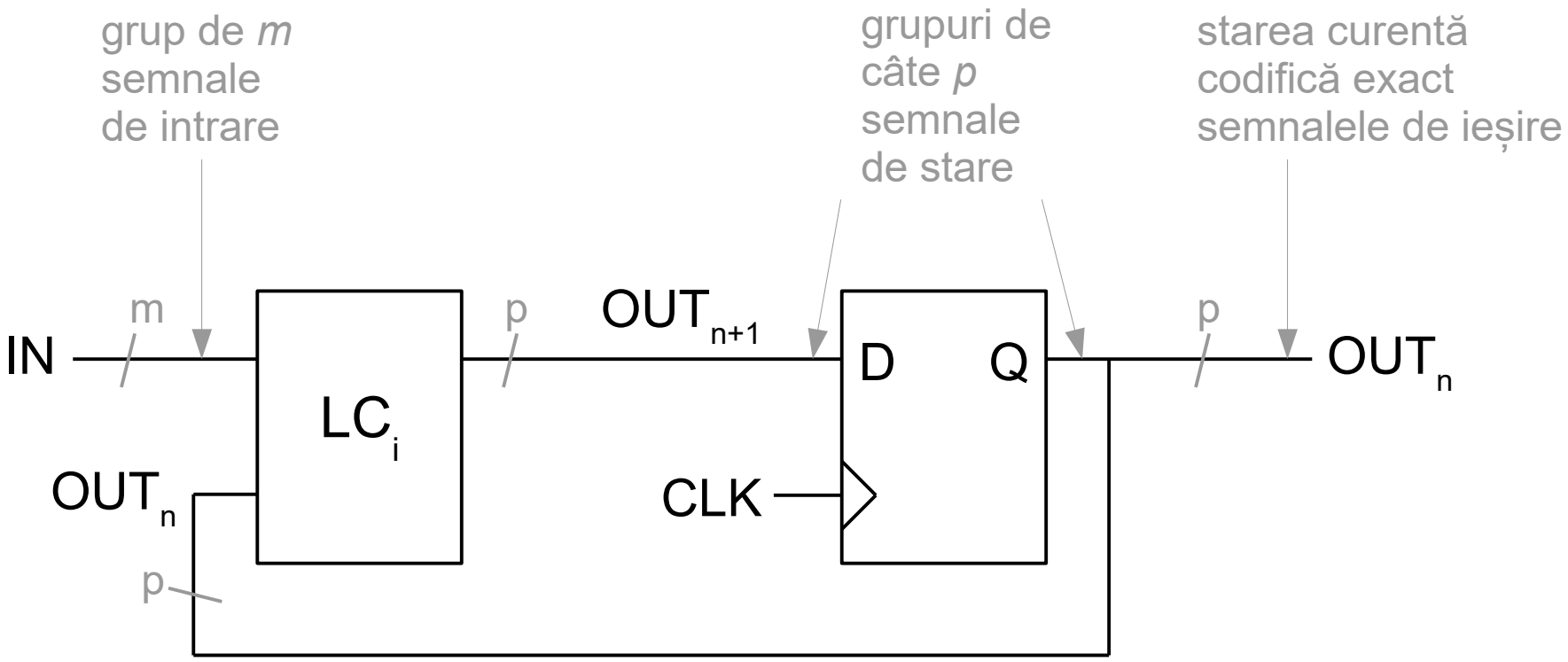
Automat de stări – schemă clasică



Automat de stări – funcționare

- pe fiecare front pozitiv al ceasului:
  - blocul logic combinațional de intrare calculează următoarea stare, pe baza stării curente și a intrărilor
  - registrul (format din bistabile D) memorează noua stare
  - blocul logic combinațional de ieșire calculează ieșirile pe baza noii stări
    - există o variantă în care și intrările intră în calculul ieșirilor (vezi automat Moore vs. automat Mealy)
- automatul nu răspunde la modificări ale intrărilor decât sincron cu ceasul (în momentul când ceasul comută din “0” în “1”) – vezi teoria bistabilului D; în acest sens semnalul de ceas temporizează o **secvență** de tranziții între stări

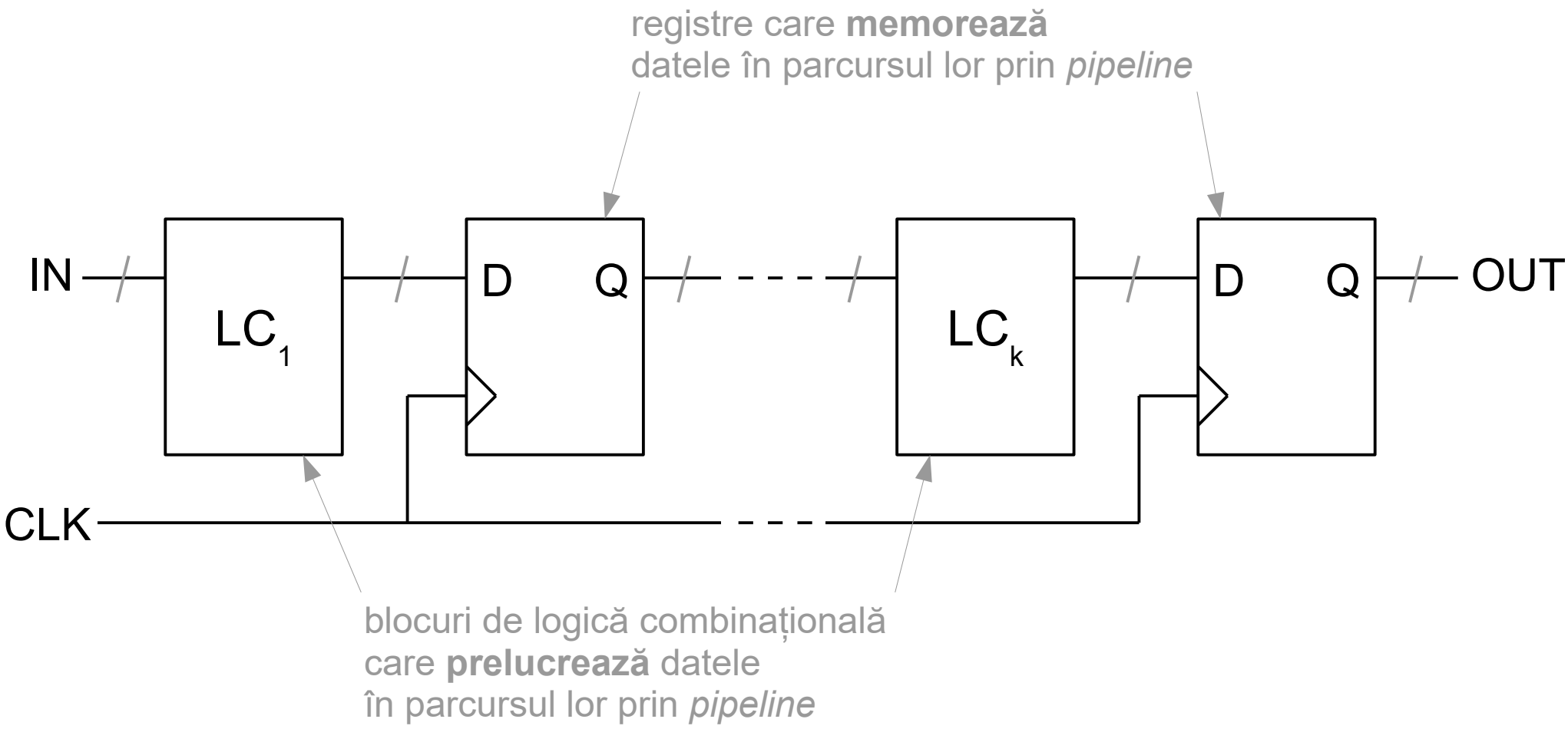
Automat de stări – ieșire directă



Variantă simplificată, în care am eliminat logica de ieșire:

* fie proiectez automatul a.î. codificarea stărilor să corespundă exact semnalelor de ieșire
* fie consider că am înglobat logica de ieșire în logica de intrare a unui alt automat similar ce s-ar conecta pe ieșirea celui discutat

Pipeline – ideal



Pipeline ideal – funcționare (1)

- La fiecare ciclu de ceas (pe frontul pozitiv al ceasului):
  - pe intrare apare un set nou de date
  - datele sunt prelucrate de primul bloc logic combinațional (LC_1)
    - fiecare bit al rezultatului apare la ieșire după un timp de propagare, dependent de calea prin care s-au propagat semnalele prin logica combinațională; după scurgerea timpului maxim de propagare considerăm că rezultatul a fost furnizat (toți biții din grup sunt disponibili pe ieșirea LC)
  - rezultatul dat de LC_1 este disponibil la intrarea în primul registru, unde va fi memorat la următorul front pozitiv al ceasului

Pipeline ideal – funcționare (2)

- Simultan cu fenomenele respective:
  - la ieșirea din primul registru apare rezultatul furnizat de LC_1 pentru datele anterioare (intrate cu un ciclu de ceas în urmă)
  - LC_2 prelucrează datele respective și le prezintă pentru a fi memorate de al doilea registru
  - și așa mai departe
- Toate blocurile combinaționale funcționează simultan (prelucrează date **în paralel**)
  - la fiecare ciclu de ceas datele avansează prin *pipeline* cu o poziție (trec spre etajul următor)

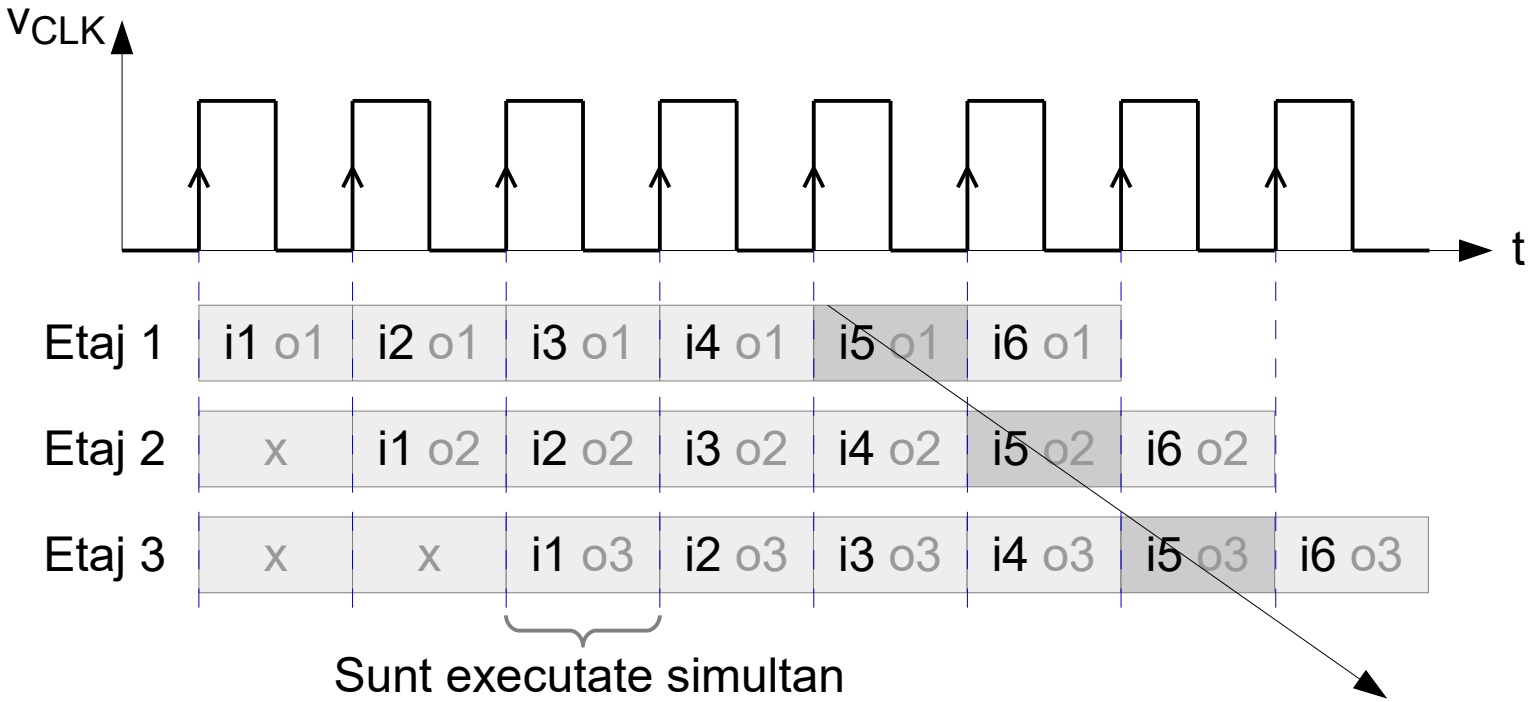
Pipeline ideal – performanțe

- Latența: perioada ceasului * numărul de etaje
  - durata de la intrarea de noi date până la producerea rezultatelor aferente pe ieșirea din *pipeline* – trebuie parcurse toate etajele combinaționale
- *Throughput*: frecvența ceasului
  - se furnizează un rezultat nou pe fiecare perioadă a ceasului

Exemplu

- Execuția unei instrucțiuni de către un procesor
  - implică realizarea mai multor etape de prelucrare, descrise prin funcții combinaționale, spre exemplu: decodificarea instrucțiunii, citirea operanzilor, execuția propriu-zisă (operația aritmetică / logică), scrierea rezultatului
  - implementare cu un singur etaj combinațional:
    - execuția instrucțiunii se realizează într-o singură etapă observabilă, într-un bloc combinațional monolitic care implementează toate sub-operațiile
  - implementare în bandă de asamblare (*pipeline*):
    - fiecare etaj din pipeline execută câte o sub-operație a unei instrucțiuni (consecutive) din program

Exemplu – *pipeline* cu 3 nivele



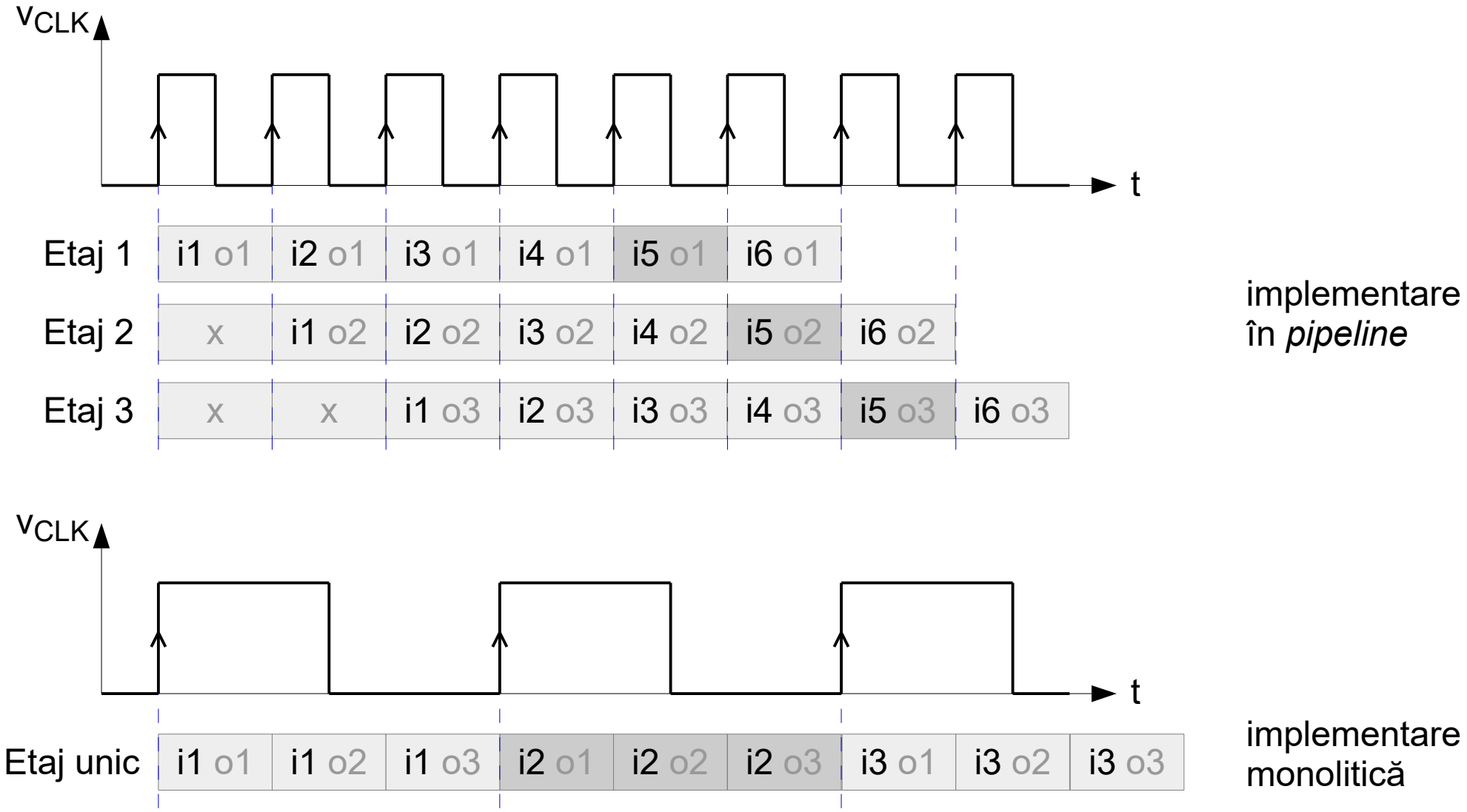
Sunt executate simultan
3 părți diferite din
3 instrucțiuni consecutive

Traseul unei instrucțiuni prin pipeline:
întârziere (*latency*) = 3 cicluri de ceas

* dar la ieșire se furnizează câte un rezultat la fiecare ciclu de ceas
throughput = frecvența ceasului

Legenda: **i***n* **o***k*
instrucțiunea *n*
(sub)**o**perația *k*

Comparație cu varianta cu un etaj



Frecvența ceasului trebuie redusă pentru a avea timp să se execute toate suboperațiile.

Comentarii (1)

- La varianta monolitică se obține (aproximativ) aceeași latență, dar un *throughput* de *n* ori mai mic (*n* = numărul de etaje din *pipeline*) deoarece trebuie crescută perioada ceasului
- Frecvența ceasului este limitată superior (perioada ceasului este limitată inferior) de:
  - timpul de propagare maxim prin blocurile logice combinaționale
  - întârzierile proprii registrelor (neglijate în discuția de față)

Comentarii (2)

- Implementarea în pipeline îmbunătățește performanța computațională (*throughput*) prin **execuția în paralel** a sub-operațiilor mai simple, versus execuția monolitică a unei operații complexe
- deoarece operațiile mai simple se traduc în scheme combinaționale de complexitate mai redusă => cu timpi de propagare mai mici
  - ideal operația trebuie împărțită în n sub-operații cu durate de execuție egale; în practică imposibil – sporul de performanță va fi mai mic ca *n.*

Observații

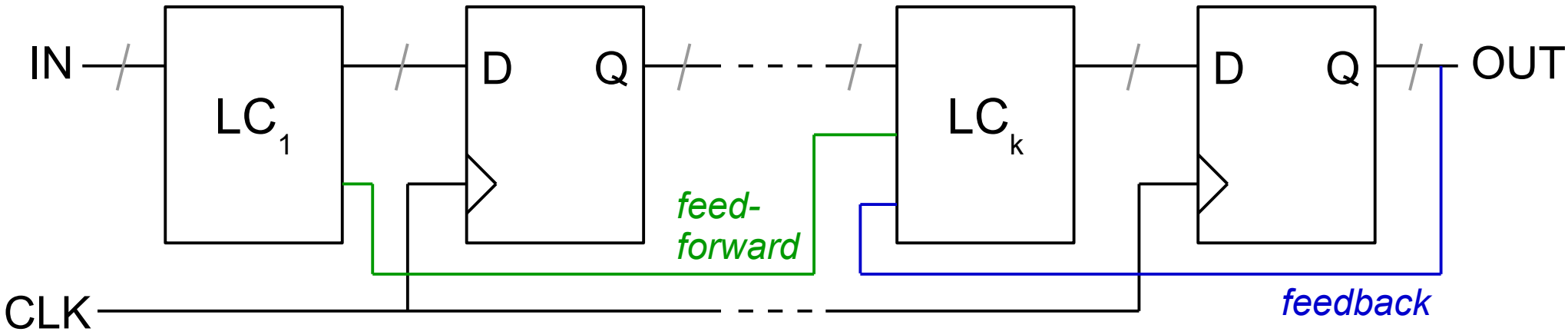
- Toate procesoarele moderne sunt implementate în *pipeline*, având între 2-3 și zeci de nivele
  - favorizează execuția instrucțiunilor consecutive – ideea fiind de a ține *pipeline*-ul plin, ocupat
  - când se execută o instrucțiune de salt condiționat (*conditional jump / branch*, corespunzătoare unui *if* din limbajele de nivel înalt) procesorul trebuie să golească pipeline-ul și să reia execuția de la noua adresă din program, ceea ce implică latență – rezultatele noi sosesc abia după propagarea noilor instrucțiuni prin *pipeline*.
  - mai multe detalii – *keywords: branch prediction*

Pipeline – real

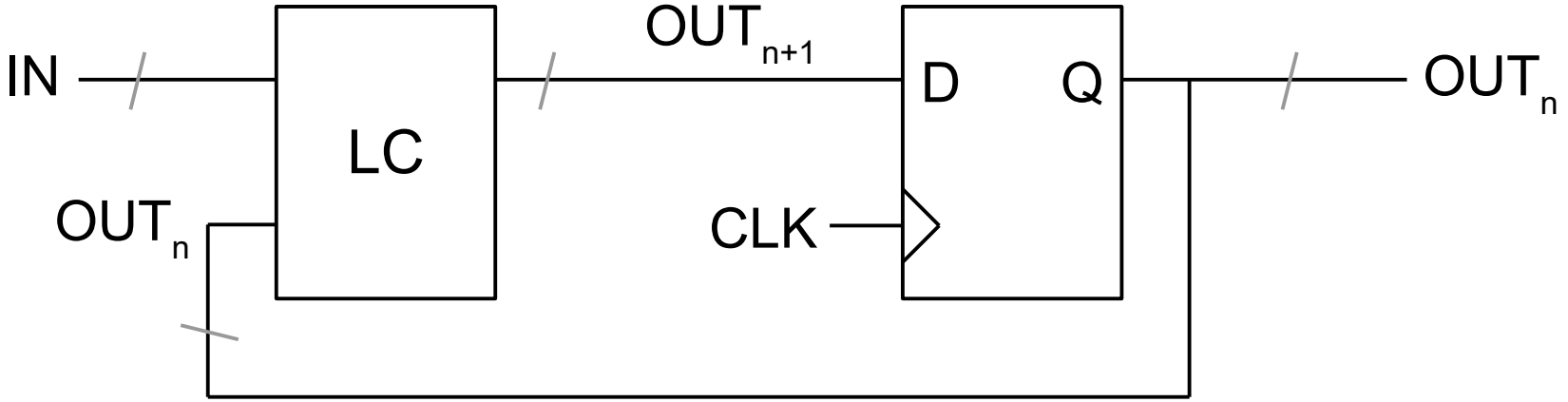
- față de situația ideală, o implementare pragmatică de pipeline poate implica și conexiuni între etaje – *feedback*, *feedforward*

- asemănare cu mașina de stări clasică ?
  - vezi planșa următoare

Pipeline vs. *State Machine*



Pipeline pragmatic



Automat de stări

Concluzii

- Atât circuitele digitale în implementare tip *pipeline* cât și cele proiectate ca automate de stări au aceeași structură fundamentală
- Sistemele de calcul sunt la nivel fundamental mașini de stări complexe (cu număr imens de stări); este mai util să fie descrise la nivel înalt – structural (schemă-bloc, vezi de ex. arhitectura generală de calculator din cursul anterior) și comportamental
- Paradigma RTL (*Register Transfer Level*) – descrierea circuitelor digitale prin blocuri combinaționale ce prelucrează date și le transferă între registre, de obicei formând structuri complexe tip *pipeline*